Learn Arduino

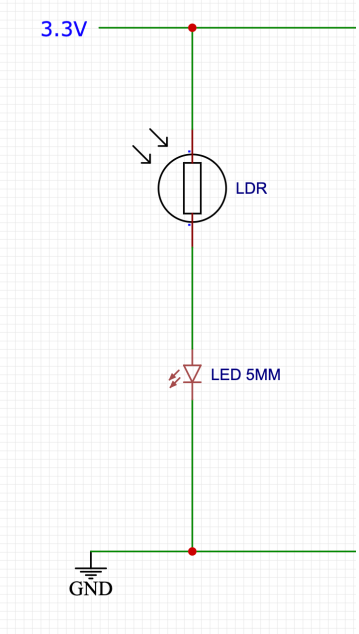
Distinctive Objects

INPUT: LDR (Light Dependent Resistor)

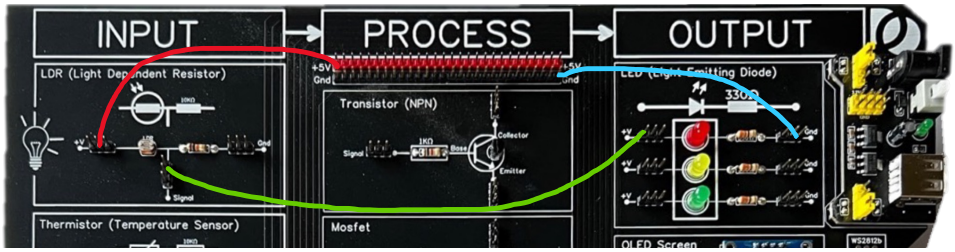
ANALOGUE INPUTS

In this guide we're going to be looking at the first of our ANALOGUE INPUTS, the LDR or Light Dependent Resistor.

To get a feel for what is going on, you're going to construct a circuit without using our microcontroller. The circuit diagram for this is shown below:

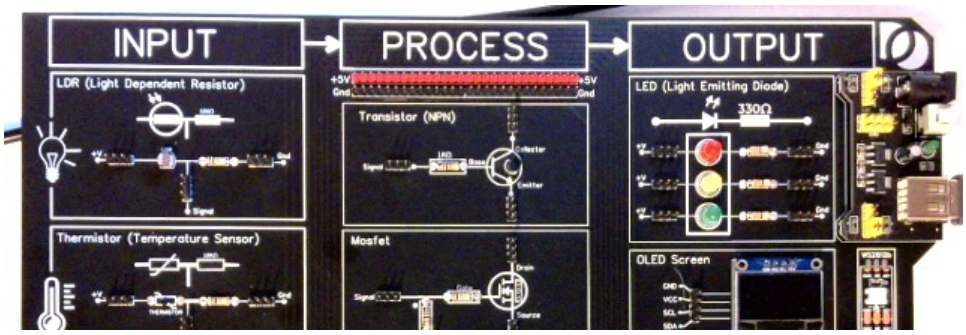


Use the 3.3V external supply for the circuit. Connect the 3.3V pin to the positive rail. From here it is just a simple series circuit, work your way from top to bottom, making sure you've made the connections as you go.



Note that you're not using the Gnd connection on the LDR and this is because there is a 10KΩ resistor along that line and we're just trying to get an idea of what is happening at different light levels.

A video demonstration

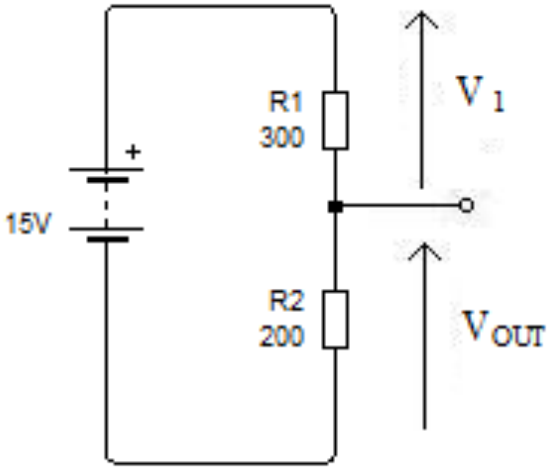


In bright light, the LDR's resistance is low, allowing more current to the LED making it brighter. In low light, the LDR's resistance is high, reducing current to the LED meaning it is dimmer.

A number of INPUTS demonstrate a similar kind of resistance change but our microcontroller senses changes in voltage.

To translate this resistance change into a voltage we use a **Voltage Divider** arrangement.

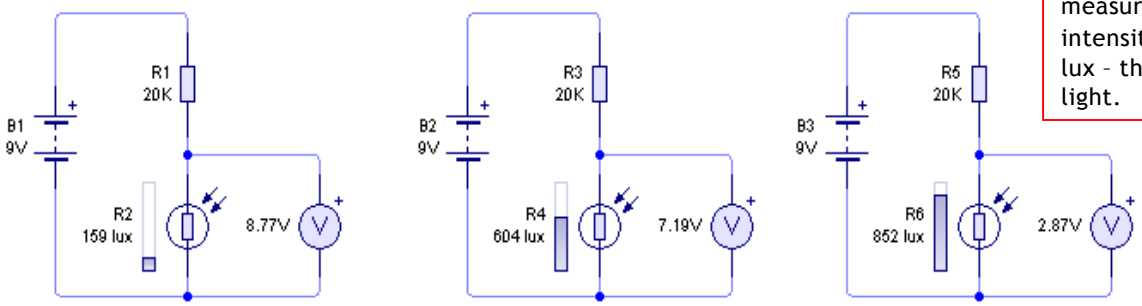
A voltage divider uses two resistors to change a voltage (at Vout on the diagram) to a required level. We'll look at how this works on the next page.



Voltage dividers.

Earlier, you looked at the characteristics of a Light Dependent Resistor (LDR). The resistance changes in proportion to the amount of light. A thermistor (temperature sensor) also works in a similar way with resistance falling as temperature rises.

This produces a very useful output when these components are connected as a voltage divider circuit. The following circuits show what happens to the voltage as the amount of light falling on an LDR increases. The V in the circuit is a voltmeter and it is measuring the voltage across the LDR.

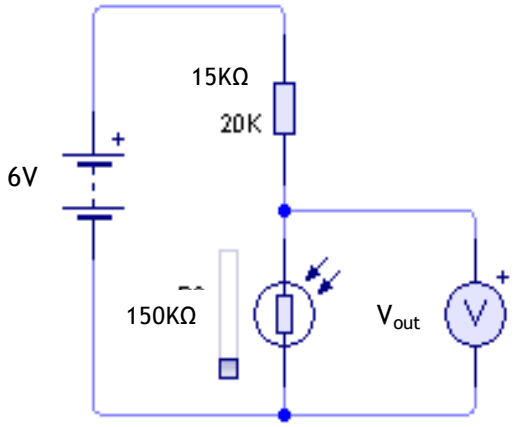


Note: Lux is a measurement of light intensity. The higher the lux - the brighter the light.

Voltage dividers - Some maths

We can actually work out what the output voltage will be mathematically as it is simply a proportion based on total resistance. We use this equation:

$$V_{out} = V_s \times \frac{R_2}{R_1 + R_2}$$



In our example on the right, at 300 lux, the resistance of our LDR is 150KΩ. The voltage at Vout is worked out like this :

$$V_{out} = V_s \times \frac{R_2}{R_1 + R_2} \quad \Rightarrow \quad V_{out} = 6 \times \frac{150}{15 + 150}$$

$$V_{out} = \underline{5.45V}$$

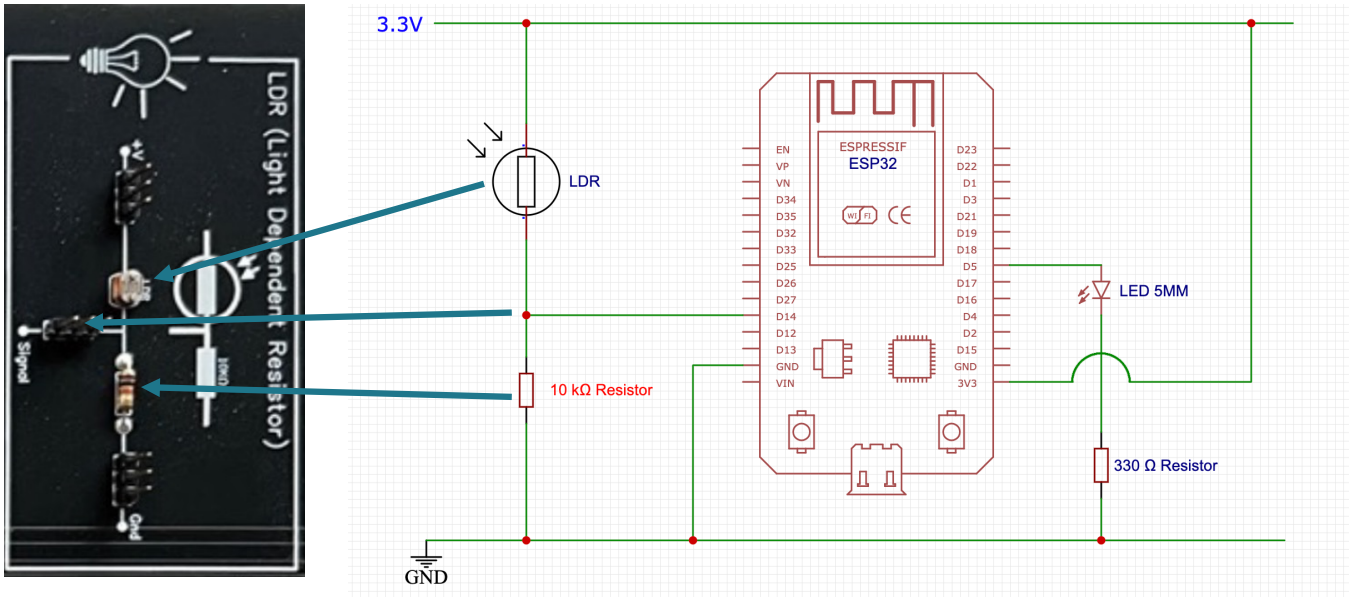
So what does this mean to our using Arduino? Well, if you now try the maths again and this time put in a lower value for R2, as would happen if the resistance of the LDR decreased in higher light levels, you see that the voltage at Vout decreases.

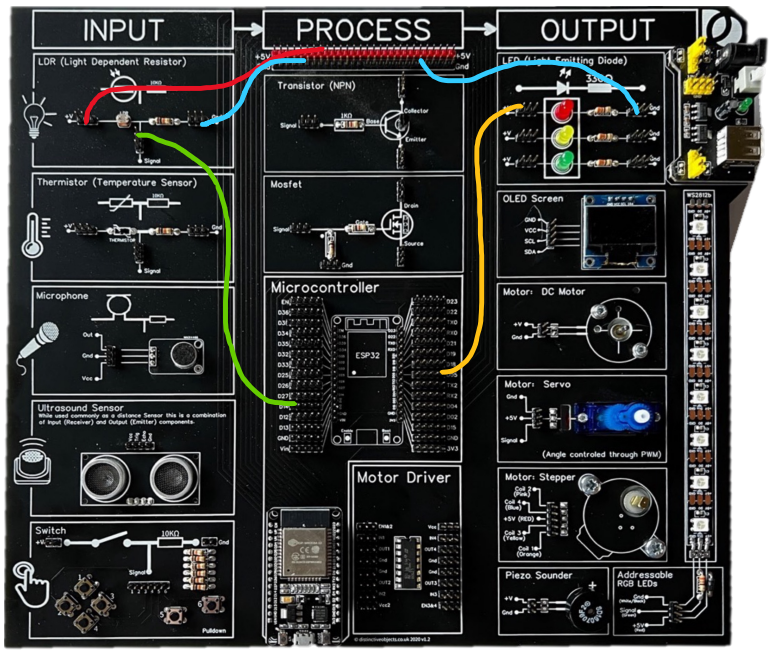
This change in voltage at the Vout line can be taken to our Arduino to be measured and used to make decisions based on external factors.

Enough theory… Let's try it!

Circuit Diagram

To construct our circuit diagram is actually fairly easy as, if you look at our Learn Arduino Board, the voltage divider aspect has already been completed for you. It's shown below so you get a feel for how this is laid out. The Vout aspect we looked at on the previous page is the same point as our Signal line on the board.





You can see here that the wiring for this is pretty simple.

To start with, you're not actually even going to be using the LED.

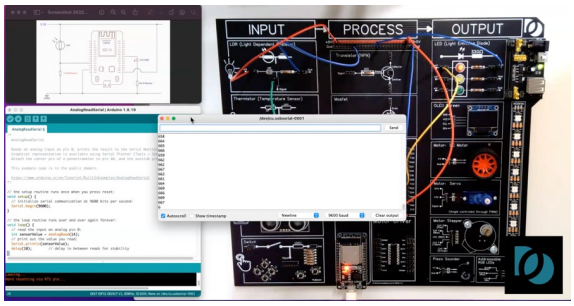
Before we even use OUTPUTS it is useful for us to get an idea of what the LDR & Voltage divider is creating at the Arduino.

For this we are going to communicate with the microcontroller while it is in use through your serial connection (You may remember right back at the beginning we chose a Serial port when linking our Arduino).

Video tutorial of wiring and code

If you prefer video to reading, head to our YouTube stream to view our walkthrough of the next steps

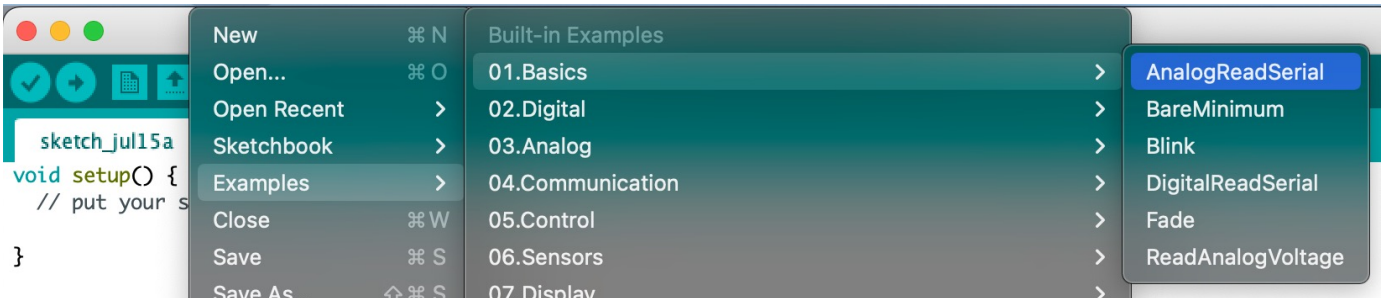
https://youtu.be/7tr9QdBywbU



The coding

Open the example AnalogReadSerial file

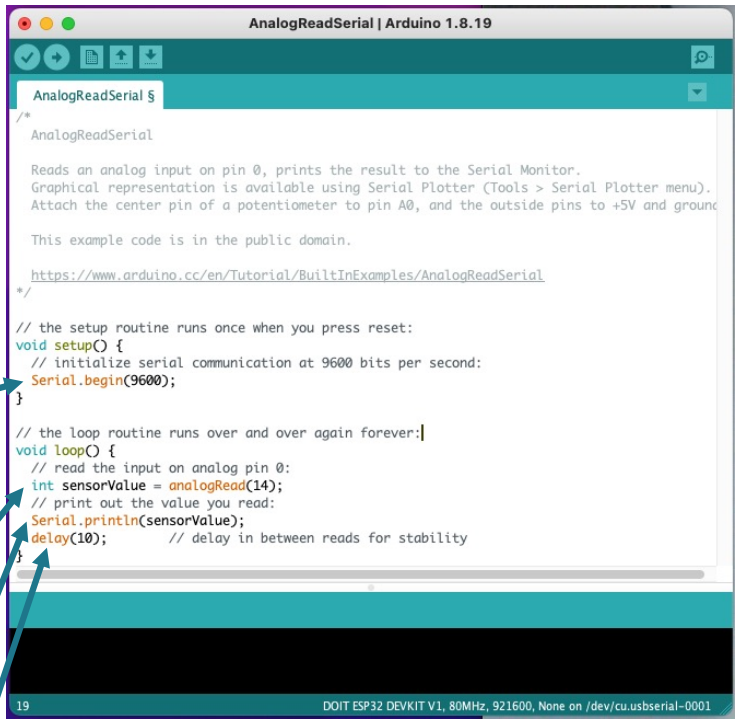
Finding AnalogReadSerial is quite simple, open the file menu – Examples - 01. Basics and select **AnalogReadSerial.**



Change the code to suit our circuit

First job, as always, is to make the code match our circuit. The AnalogReadSerial code only really needs two small changes

- analogRead(A0); to analogRead(14)
- Change the **delay(1);** number to **10.**



Let's consider what's going on

Serial.begin(9600); this command asks our microcontroller to open up a communication channel with the computer. The 9600 aspect is just the speed at which they are going to communicate (needs to be the same on the computer but more of that later)

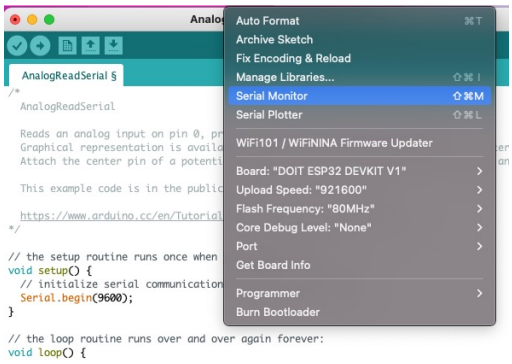
Here we create a new variable called SensorValue and make it the value of whatever is coming into pin 14. int simply means we want it to hold the whole numbers (none of those pesky decimals!)

Serial.print tells the microcontroller to send the number held in the SensorValue variable to the computer so we can see it

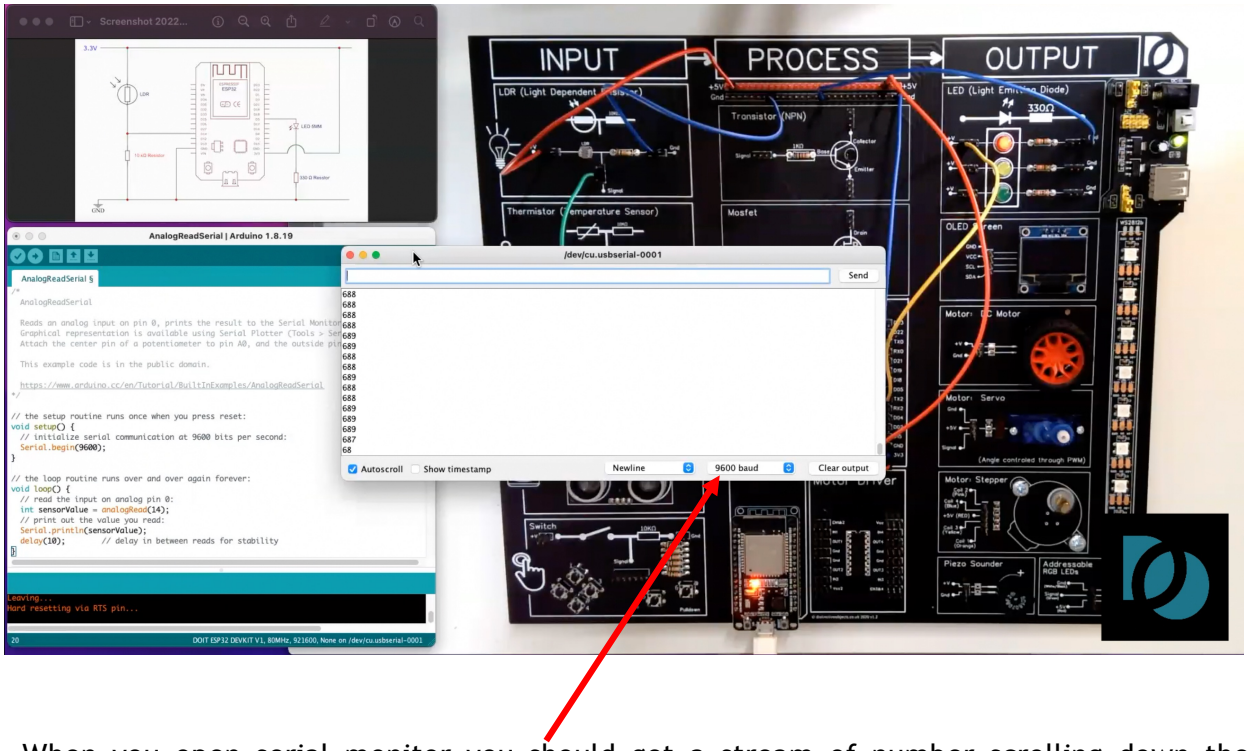
Our delay is there to just put a short pause in before the sketch restarts to give us a chance to see what is going on. The ln part just means that each value is shown on its own line. Without this it would just be a huge stream of continuous numbers.

Upload and open Serial Monitor

Go to the main toolbar, choose **Tools - Serial Monitor**



Serial Monitor



When you open serial monitor you should get a stream of number scrolling down the screen. This should range from 0-4056. If you haven't got this stream of numbers the first thing to check is the baud rate, make sure this is set to 9600 as this is what the sketch set up and the computer and microcontroller will not communicate if this is different. It often shows as a few different characters or a backward question mark.

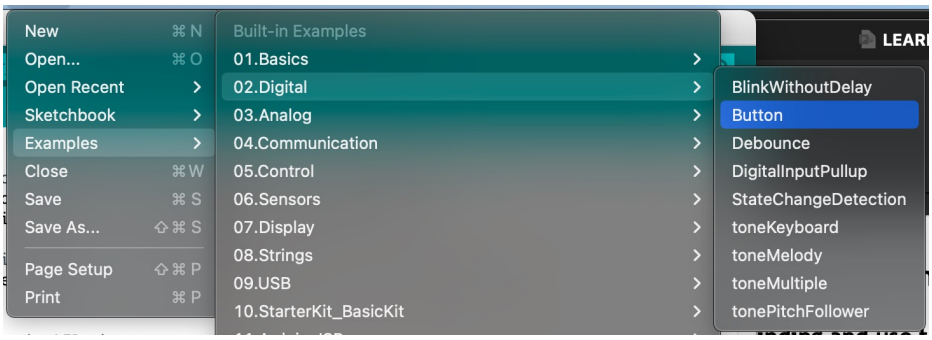
Now try covering the LDR, you should see the numbers change, in darker environments the numbers should be lower, brighter the numbers will be higher.

<u>Take a note of the most common number - in the example above it was about 650</u>

Streetlamp.

Okay, so there's a stream of numbers - so what right? Well if we can sense the environment we can react to it. To finish this Learn section we're going to use these numbers to make a simple simulation of a streetlamp.

What we need to do to make this work is to add a decision to our serialRead sketch. To make this easier we're going to re-open the button sketch we use from examples when we looked at using buttons and use the decision process from this. Re-open Button from the examples menu:



Re-purposing some code

Look at the button code on the right - there are key parts that we can take from this.

<u>Remember when copying, do not duplicate the void setup() or void loop() as this doesn't compile.</u>

You will need to adjust the code as we did earlier e.g. the ledPin changed to 5 so it matches our circuit and the if statement also needs to be changed to sensorValue as this is what is measures in the AnalogRead sketch!

AnalogReadSerial | Arduino 1.8.19

AnalogReadSerial §

```
/*
  AnalogReadSerial

  Reads an analog input on pin 0, prints the result to the Serial Monitor.
  Graphical representation is available using Serial Plotter (Tools > Serial Plotter menu).
  Attach the center pin of a potentiometer to pin A0, and the outside pins to +5V and ground

  This example code is in the public domain.

  https://www.arduino.cc/en/Tutorial/BuiltInExamples/AnalogReadSerial
*/

// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(14);
  // print out the value you read:
  Serial.println(sensorValue);
  delay(10);        // delay in between reads for stability
}
```

19 DOIT ESP32 DEVKIT V1, 80MHz, 921600, None on /dev/cu.usbserial-0001

Button | Arduino 1.8.19

Button §

```
const int ledPin =  13;      // the number of the pushbutton pin
                             // the number of the LED pin

// variables will change:
int buttonState = 0;         // variable for reading the pushbutton status

void setup() {
  // initialize the LED pin as an output:
  pinMode(ledPin, OUTPUT);
  // initialize the pushbutton pin as an input:
  pinMode(buttonPin, INPUT);
}

void loop() {
  // read the state of the pushbutton value:
  buttonState = digitalRead(buttonPin);

  // check if the pushbutton is pressed. If it is, the buttonState is HIGH:
  if (buttonState == HIGH) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
  }
}
```

Done uploading.

Finished Code

AnalogReadSerial | Arduino 1.8.19

AnalogReadSerial §

```
const int ledPin =  13;

// the setup routine runs once when you press reset:
void setup() {
  // initialize serial communication at 9600 bits per second:
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
  // read the input on analog pin 0:
  int sensorValue = analogRead(14);
 if (sensorValue <300) {
    // turn LED on:
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off:
    digitalWrite(ledPin, LOW);
  }

  // print out the value you read:
  Serial.println(sensorValue);
  delay(10);        // delay in between reads for stability
}
```

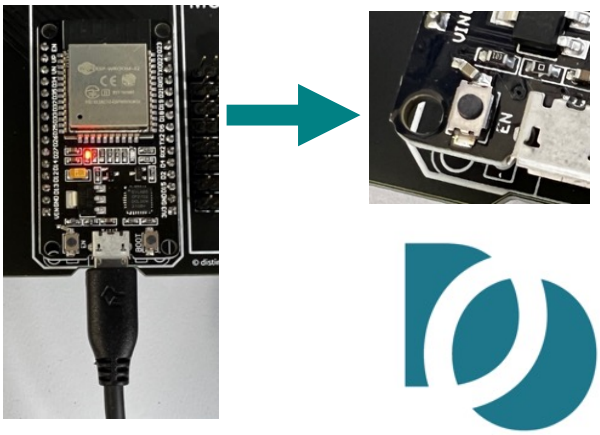
Done compiling.

```
Sketch uses 239217 bytes (18%) of program storage space. Maximum is 13
Global variables use 16508 bytes (5%) of dynamic memory, leaving 31117
```

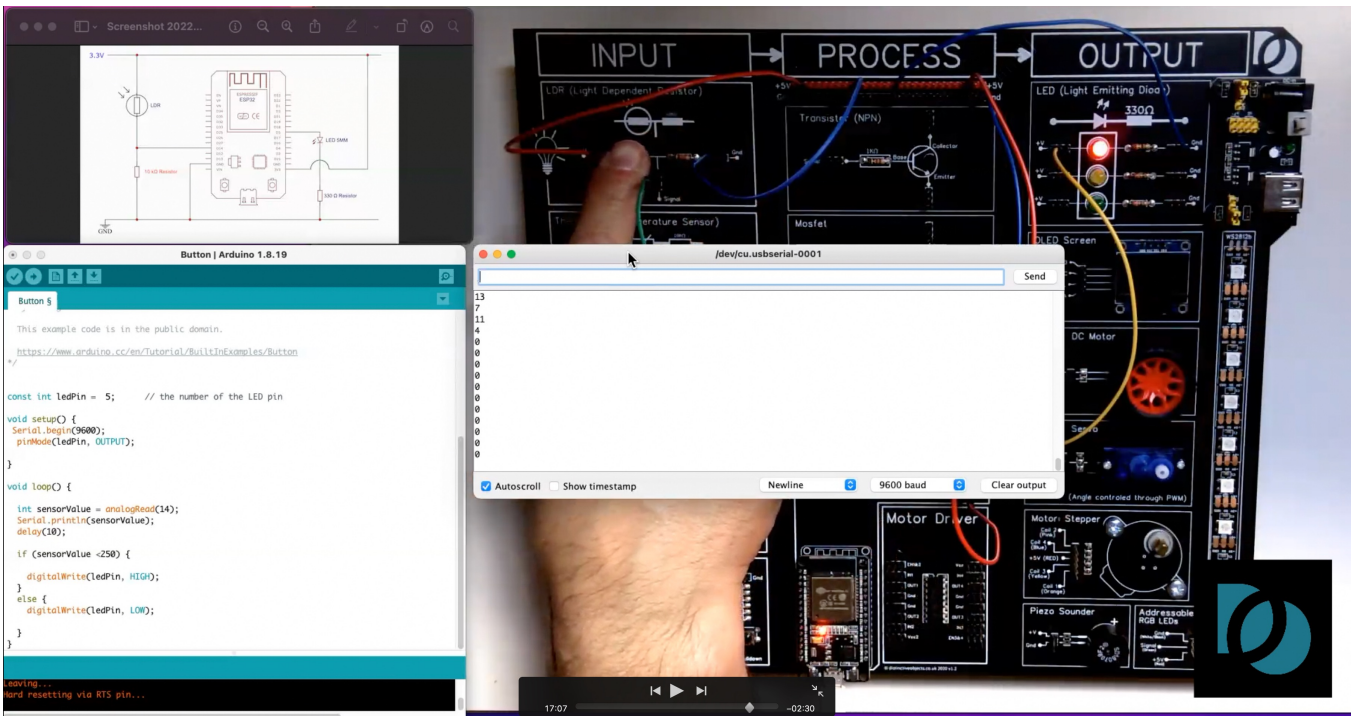
1 DOIT ESP32 DEVKIT V1, 80MHz, 921600, None on /dev/cu.usbserial-0001

There is another change that needs to be made to the code and it concerns the fact that the LDR is an analogue INPUT. It will not really be either HIGH or LOW but is, as we have seen, a value. We are going to change this to <u><300</u> so the LED lights when it gets dark

Upload your code
Upload your code



Success!



What if it didn't work?

There are a few things to check if it isn't doing what you expect. The most common is a compilation failure and this is almost always to do with a missing semi colon or the line hasn't been put in the right place.

If it uploads but doesn't work, check your Serial Monitor, if that is still showing different numbers then did you change your If statement to read sensorValue rather than buttonPin?

Lastly, check the inequality (in other words the <300 bit of the If statement). Are the numbers in your serial monitor going down that far? If not, change the value to be similar to what you are seeing on your serial monitor. For example if it only ever goes as low at 600 then change your <300 to read something like <700.

Have a go

Something to have a go at. See if you can get the LED to be brighter in brighter lights. This seems really odd but if you think about your phone screen the backlight does exactly this to make sure you can still read the screen in bright sunlight.

So what needs to change on the sketch to get this to change its behaviour. Have a go and if you get stuck check out the video as the solution is on there!

Copyright Distinctive Objects Ltd 2022